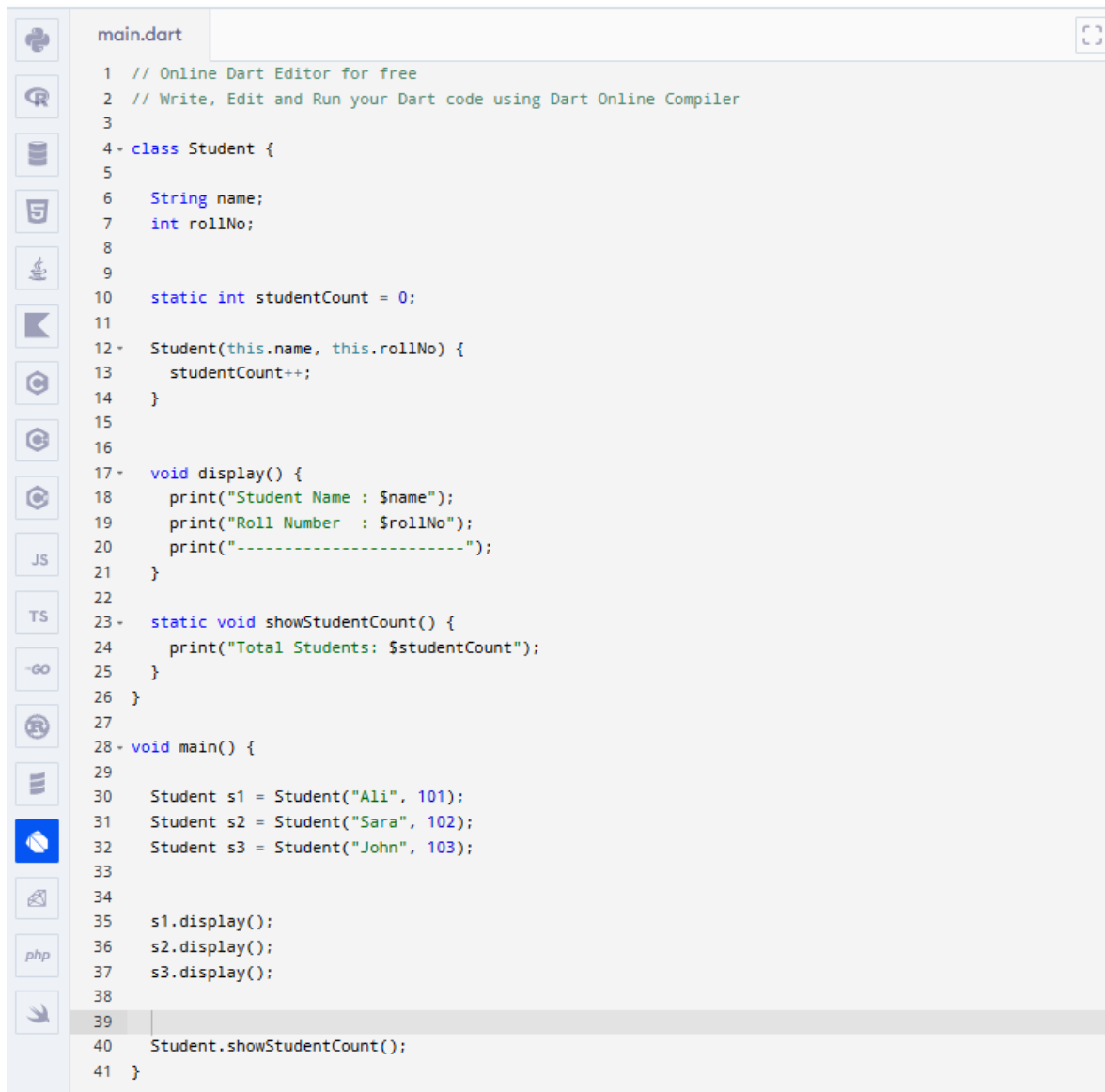


APP DEVELOPMENT IN FLUTTER

IMPLEMENT OBJECT ORIENTED CONCEPTS IN DART:

3A) IMPLEMENT SIMPLE CLASS IN DART AND ALSO
SHOW THE USAGE OF SATIC NUMBER.

CODE:



```
main.dart
1 // Online Dart Editor for free
2 // Write, Edit and Run your Dart code using Dart Online Compiler
3
4 class Student {
5
6     String name;
7     int rollNo;
8
9
10    static int studentCount = 0;
11
12    Student(this.name, this.rollNo) {
13        studentCount++;
14    }
15
16
17    void display() {
18        print("Student Name : $name");
19        print("Roll Number : $rollNo");
20        print("-----");
21    }
22
23    static void showStudentCount() {
24        print("Total Students: $studentCount");
25    }
26 }
27
28 void main() {
29
30     Student s1 = Student("Ali", 101);
31     Student s2 = Student("Sara", 102);
32     Student s3 = Student("John", 103);
33
34
35     s1.display();
36     s2.display();
37     s3.display();
38
39
40     Student.showStudentCount();
41 }
```

OUTPUT:

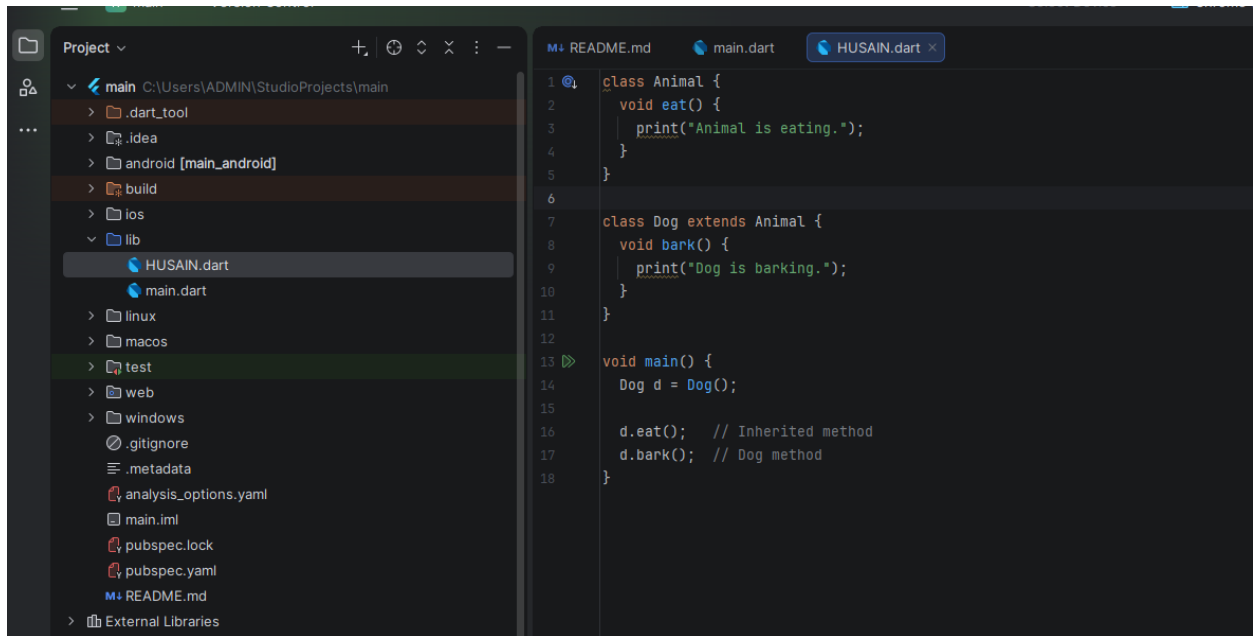
MOHAMAD HUSAIN KOLAD
TY BVOC-SDPRACTICAL 3 [Type here]

Output

```
Student Name : Ali  
Roll Number  : 101  
-----  
Student Name : Sara  
Roll Number  : 102  
-----  
Student Name : John  
Roll Number  : 103  
-----  
Total Students: 3  
  
=== Code Execution Successful ===
```

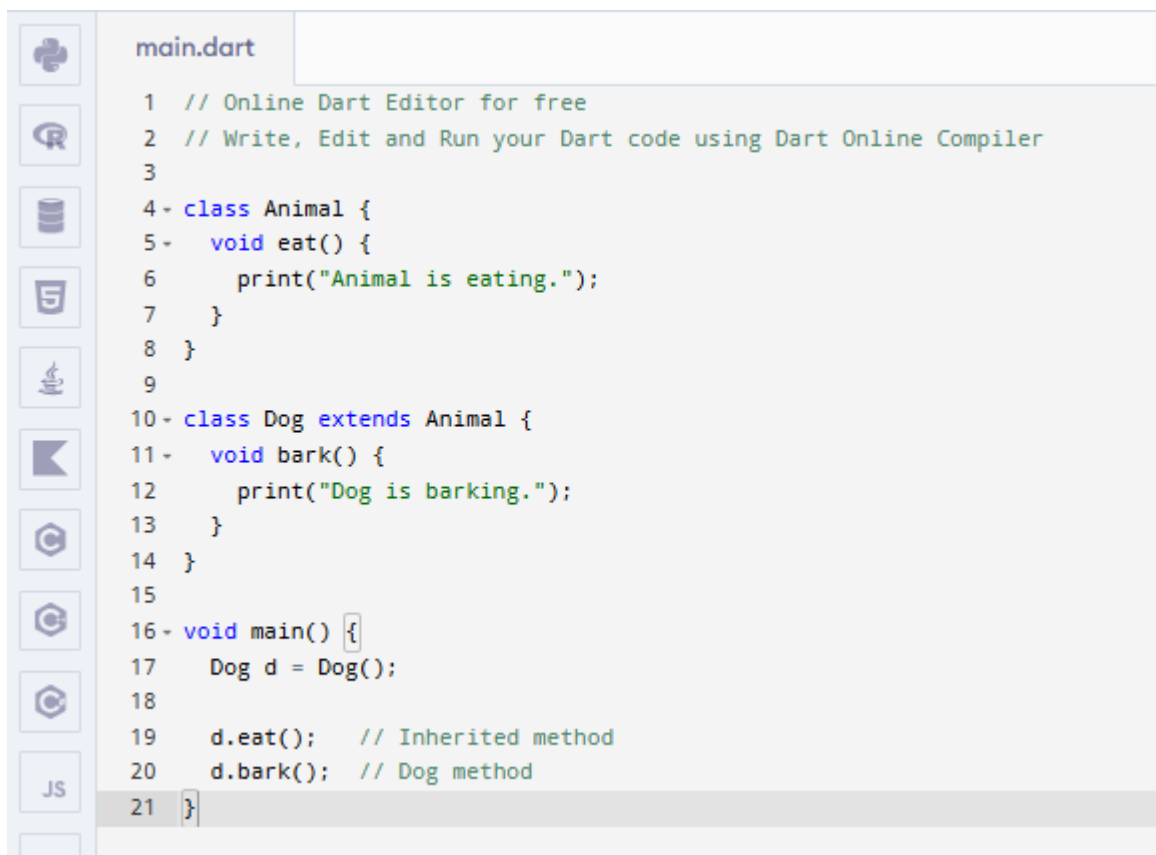
3B) IMPLEMENT SINGLE INHERITENCE
CODE:

MOHAMAD HUSAIN KOLAD
TY BVOC-SDPRACTICAL 3 [Type here]



The screenshot shows an IDE with a project named 'main' located at 'C:\Users\ADMIN\StudioProjects\main'. The file explorer on the left shows the project structure, including folders like '.dart_tool', '.idea', 'android', 'build', 'ios', 'lib', 'linux', 'macos', 'test', 'web', 'windows', and files like 'analysis_options.yaml', 'main.iml', 'pubspec.lock', 'pubspec.yaml', 'README.md', and 'External Libraries'. The 'lib' folder is expanded, showing 'HUSAIN.dart' and 'main.dart'. The 'HUSAIN.dart' file is selected, and its content is displayed in the editor. The code defines an 'Animal' class with an 'eat()' method, a 'Dog' class that extends 'Animal' with a 'bark()' method, and a 'main()' function that creates a 'Dog' object and calls both methods.

```
1 class Animal {  
2   void eat() {  
3     print("Animal is eating.");  
4   }  
5 }  
6  
7 class Dog extends Animal {  
8   void bark() {  
9     print("Dog is barking.");  
10  }  
11 }  
12  
13 void main() {  
14   Dog d = Dog();  
15  
16   d.eat(); // Inherited method  
17   d.bark(); // Dog method  
18 }
```



The screenshot shows an online Dart editor interface. The file explorer on the left has icons for various languages and frameworks, including Dart, JavaScript, and others. The 'main.dart' file is selected, and its content is displayed in the editor. The code is identical to the one in the IDE screenshot, defining an 'Animal' class, a 'Dog' class, and a 'main()' function.

```
1 // Online Dart Editor for free  
2 // Write, Edit and Run your Dart code using Dart Online Compiler  
3  
4 class Animal {  
5   void eat() {  
6     print("Animal is eating.");  
7   }  
8 }  
9  
10 class Dog extends Animal {  
11   void bark() {  
12     print("Dog is barking.");  
13   }  
14 }  
15  
16 void main() {  
17   Dog d = Dog();  
18  
19   d.eat(); // Inherited method  
20   d.bark(); // Dog method  
21 }
```

OUTPUT:

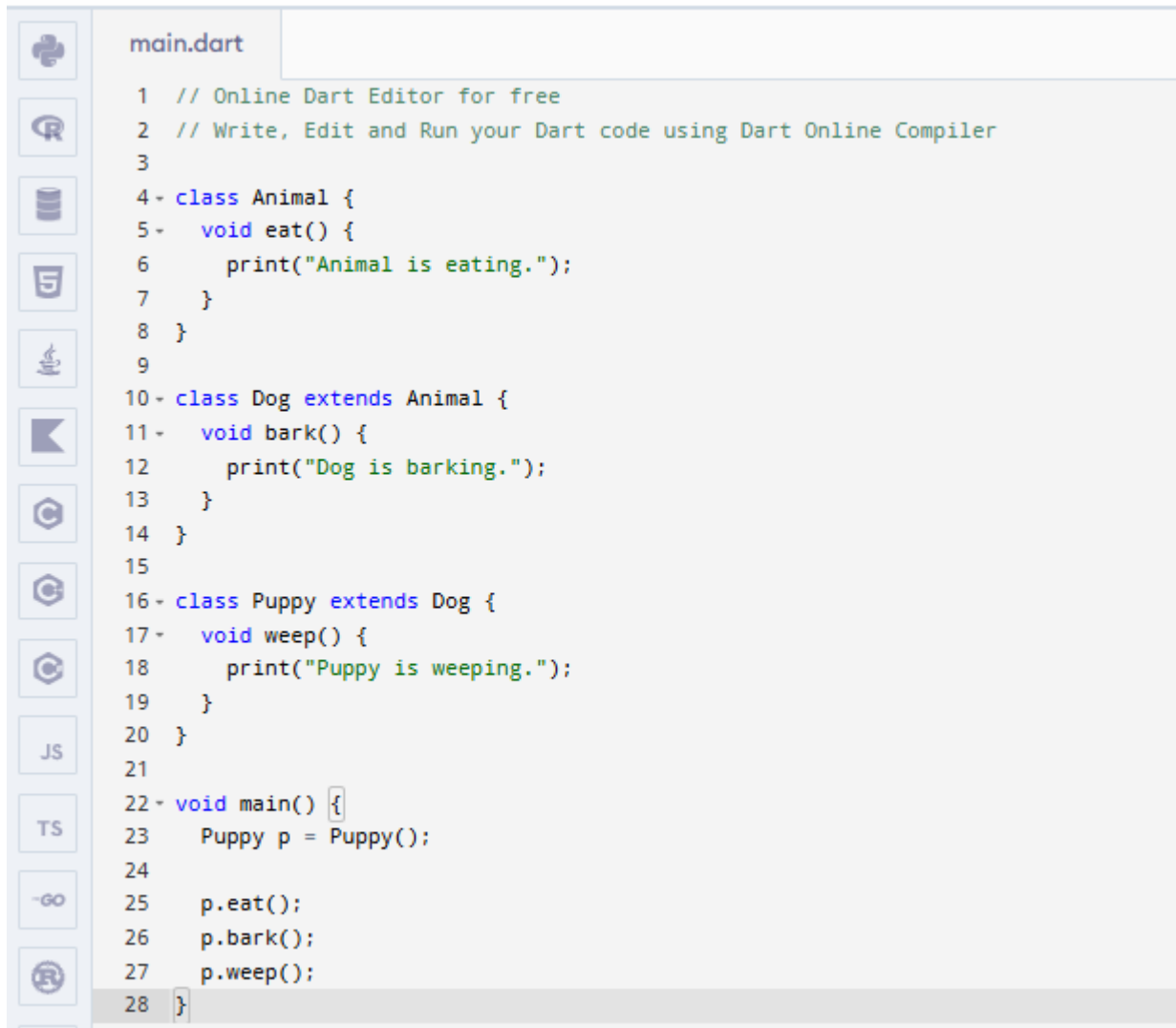
MOHAMAD HUSAIN KOLAD
TY BVOC-SDPRACTICAL 3 [Type here]

```
Output  
  
Animal is eating.  
Dog is barking.  
  
=== Code Execution Successful ===
```



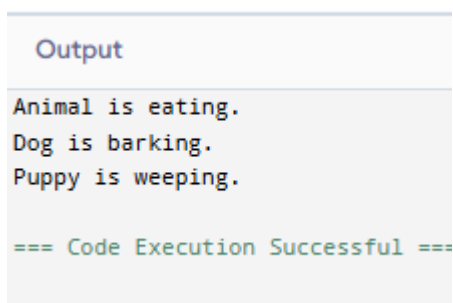
3C) IMPLEMENT MULTILEVEL INHERITENCE

CODE:



```
main.dart
1 // Online Dart Editor for free
2 // Write, Edit and Run your Dart code using Dart Online Compiler
3
4 class Animal {
5   void eat() {
6     print("Animal is eating.");
7   }
8 }
9
10 class Dog extends Animal {
11   void bark() {
12     print("Dog is barking.");
13   }
14 }
15
16 class Puppy extends Dog {
17   void weep() {
18     print("Puppy is weeping.");
19   }
20 }
21
22 void main() {
23   Puppy p = Puppy();
24
25   p.eat();
26   p.bark();
27   p.weep();
28 }
```

OUTPUT:



```
Output
Animal is eating.
Dog is barking.
Puppy is weeping.

=== Code Execution Successful ===
```

3D) MULTIPLE INTERFACE INHTERFACE

CODE:

```
main.dart
1 // Online Dart Editor for free
2 // Write, Edit and Run your Dart code using Dart Online Compiler
3
4 class Father {
5   void showFather() {
6     print("Father's Method");
7   }
8 }
9
10 class Mother {
11   void showMother() {
12     print("Mother's Method");
13   }
14 }
15
16 class Child implements Father, Mother {
17   @override
18   void showFather() {
19     print("Father's Method");
20   }
21
22   @override
23   void showMother() {
24     print("Mother's Method");
25   }
26
27   void showChild() {
28     print("Child's Method");
29   }
30 }
31
32 void main() {
33   Child c = Child();
34
35   c.showFather();
36   c.showMother();
37   c.showChild();
38 }
```

OUTPUT:

```
Output
Father's Method
Mother's Method
Child's Method

=== Code Execution Successful ===
```

3E) IMPLEMENT THE CONCEPT OF MIXIN

CODE:

```
main.dart
1 // Online Dart Editor for free
2 // Write, Edit and Run your Dart code using Dart Online Compiler
3
4+ mixin CanRun {
5+   void run() {
6       print("Running...");
7   }
8 }
9
10+ mixin CanJump {
11+   void jump() {
12       print("Jumping...");
13   }
14 }
15
16+ class Human with CanRun, CanJump {
17+   void display() {
18       print("Human Actions");
19   }
20 }
21
22+ void main() {
23     Human h = Human();
24
25     h.display();
26     h.run();
27     h.jump();
28 }
```

OUTPUT:

```
Output
Human Actions
Running...
Jumping...

=== Code Execution Successful ===
```